

KARNATAK LAW SOCIETY'S
GOGTE INSTITUTE OF TECHNOLOGY

UDYAMBAG, BELAGAVI-590008

(An Autonomous Institution under Visvesvaraya Technological University, Belagavi)

(APPROVED BY AICTE, NEW DELHI)



*Report On
project of the title*

***“Word Count and Vertical Histogram Generation Using File Handling in
C on Linux”***

*Submitted in the partial fulfillment for the academic requirement of
6th Semester B.E.*

*In
Electronics & Communication Engineering*

SUBMITTED BY:

NAME	USN	Marks
Abhay S Nagure	2GI24EC400	
Abhishek P Karadi	2GI24EC402	
Vinyas V Kulkarni	2GI24EC417	

GUIDED BY:
Prof. Anusha Hodlur
Associate Professor, GIT
2025-2026

--

Karnataka Law Society's
GOGTE INSTITUTE OF TECHNOLOGY
Udyambag Belagavi -590008
Karnataka, India.
Department of Electronics & Communication and Engineering



Certificate

This is to certify that the project titled **“Word Count and Vertical Histogram Generation Using File Handling in C on Linux ”** carried out by Abhay S Nagure, Abhishek P Karadi, Vinyas V Kulkarni **USNs: 2GI24EC400, 2GI24EC402, 2GI24EC417** have submitted in partial fulfilment of the requirements for 6th semester B.E. in **ELECTRONICS AND COMMUNICATION AND ENGINEERING**. It is certified that all corrections/suggestions indicated have been incorporated in the report. The seminar report has been approved as it satisfies the academic requirements prescribed for the said degree.

Signature of the H.O.D
Prof. Supriya Shanbhag

Signature of the guide
Prof. Anusha Hodlur

Table Of Contents

Chapter No.	Title
1	Abstract
2	Introduction
3	Objectives
4	Methodology
5	Implementation
6	Results
7	Conclusion
8	References

1. Abstract:

This report presents the development of a Linux-based command-line utility written in the C programming language for text file analysis and word statistics generation. The primary objective of this utility is to read a user-specified file through command-line arguments, count the total number of words, classify words based on their lengths, and display a vertical histogram using the “*” character.

The application follows a sequential file-processing approach, reading words directly from the input file without storing the complete file in memory, thereby ensuring efficient execution with $O(n)$ time complexity and minimal memory usage. Word lengths are calculated using standard string-handling functions and stored in a fixed-size array for frequency analysis.

The software was compiled and executed in a Linux environment using the **GNU Compiler Collection (gcc)**. Experimental validation confirms accurate word counting, word-length classification, and proper histogram generation, making the utility an efficient solution for text analysis and file-processing operations in C.

2. Introduction:

Text Analysis and Word-Length Distribution in Digital Documents

In modern computing, text files serve as a fundamental medium for storing, exchanging, and processing information across software systems. Analyzing textual data efficiently is essential in applications such as document processing, linguistic analysis, content management, and data mining. One of the core challenges in text analysis is determining statistical properties of textual content, such as total word count and the frequency distribution of word lengths, which provide meaningful insights into document structure and readability.

Manual analysis of text files becomes impractical and error-prone when dealing with large datasets or multiple documents. Therefore, automated text-processing utilities are widely adopted to perform rapid and accurate file analysis. A command-line based solution enables direct file access, efficient execution, and integration with Linux-based environments commonly used in software development and system programming.

Evolution of File Processing Utilities

Modern operating systems rely heavily on lightweight text-processing tools for analyzing and transforming file contents. Following the UNIX philosophy of creating efficient single-purpose utilities, file-analysis programs are designed to process input streams systematically while maintaining low memory consumption and predictable execution behavior.

Implementing such utilities in the **C programming language** provides direct access to low-level file handling, efficient string manipulation, and high execution speed with minimal runtime overhead. By leveraging standard file I/O operations and array-based frequency tracking, the developed utility efficiently computes total word count, categorizes words according to length, and generates a vertical histogram for visual representation of word distribution in Linux environments.

3. Objectives

- **Dynamic CLI File Input:** The program must accept the target file name through command-line arguments (`argv`) instead of using hardcoded file paths.
- **Word Count Computation:** The system must read the complete file and accurately count the total number of words present.
- **Word Length Classification:** The program must determine the length of each word and maintain a count of words based on character length.
- **Vertical Histogram Generation:** The utility must generate a vertical histogram using the “*” character to visually represent the frequency distribution of word lengths.
- **Console Output Display:** Results including total word count, word-length frequency, and histogram must be displayed through the standard output stream (`stdout`).

Non-Functional Requirements

- **Constant Space Complexity ($O(1)$ Memory Usage):** The application must process files efficiently using fixed-size storage for word-length counts without loading the complete file into memory.
- **Linear Time Complexity ($O(n)$ Processing):** The algorithm must scan the input file in a single pass, ensuring execution time scales linearly with file size.
- **Fault-Tolerant File Handling:** The program must validate command-line arguments and handle invalid or inaccessible files gracefully by displaying appropriate error messages instead of terminating unexpectedly.
- **Linux Compatibility:** The utility must compile and execute reliably in a Linux environment using the GNU Compiler Collection (`gcc`).

4. Methodology:

Sequential File Processing vs. Complete File Storage Models
During the architectural design phase of the utility, two distinct processing approaches were considered: Complete File Storage and Sequential File Processing.

Engineering Metric	Complete File Storage Model	Sequential File Processing Model
Memory Allocation	Dynamic memory for entire file	Fixed-size variable allocation
Space Complexity	$O(n)$ where n is file size	$O(1)$ Constant Memory
File Size Handling	Limited by available RAM	Suitable for large files
Processing Method	Entire file loaded before analysis	Word-by-word sequential scanning
Implementation Complexity	High (memory management required)	Low (simple file traversal)

The **Sequential File Processing Model** was selected for this utility. By reading words directly from the file using standard file-handling functions, the program minimizes memory usage and avoids storing the entire file content, ensuring efficient execution and reliable performance even for large text files.

4.2 Processing Logic Representation

The core logic of the word analysis application can be represented through a simple sequential processing model containing two primary operations.

1. **Read Word State:** The program reads each word from the input file sequentially using file input functions. If a valid word is detected, the system increments the total word counter and proceeds to evaluate its length.
2. **Word Analysis State:** The program calculates the word length using string functions, updates the corresponding frequency count in the array, and continues processing until the end of the file is reached.

Selection of Fixed-Size Frequency Array

An important implementation decision in this utility was the use of a fixed-size integer array for storing word-length frequencies instead of dynamically allocating memory during execution. Since word lengths remain within practical limits, a predefined array enables direct indexing and faster access while reducing computational overhead.

By storing only the count of word lengths rather than preserving the complete file content, the program maintains constant auxiliary memory usage and avoids unnecessary memory allocation complexity. This design ensures efficient file traversal, accurate word statistics computation, and reliable histogram generation in Linux-based systems.

5.Implementation:

1. Header Incorporations and Memory Definitions

- **#include <stdio.h> (Standard Input/Output):**

Imports core file-handling and display functions such as `fopen()`, `fscanf()`, `printf()`, and `fclose()`, which enable file reading and result visualization.

- **#include <string.h> (String Handling):**

Provides access to string-processing functions such as `strlen()`, used to determine the length of each word read from the file.

2. Variable Registration Frame

- **The File Pointer (FILE *fp):**

Declared to establish a connection with the input file. It acts as a control reference for reading textual content from the specified file.

- **The Word Buffer (`char word[100]`):**

Stores individual words extracted from the input file during sequential scanning.

- **Word Counter (`int word_count`):**

Maintains the total number of words processed from the file.

- **Length Frequency Array (`int length_count[50]`):**

Stores the occurrence count of words based on their character length.

- **Supporting Variables (`length, i, j, max`):**

Used for word-length calculation, loop execution, and histogram generation.

Step 1: Command-Line Interface (CLI) Boundary Controls

- **Argument Verification:**

The condition `if (argc != 2)` validates the execution structure by ensuring that exactly one filename argument is supplied through the command line (`argv[1]`).

- **Error Message Handling:**

If the required file argument is missing or incorrect, the program displays a usage message through `printf()` to guide proper execution.

- **Graceful Program Exit:**

The statement `return 1;` immediately terminates execution when invalid arguments are detected.

Step 2: File System Resource Binding and Validation

- **Read-Only File Initialization (`fopen`):**

The statement `fopen(argv[1], "r")` opens the target file in read mode for content extraction.

- **Null Pointer Validation:**

The condition `if (fp == NULL)` checks whether the file was successfully opened or rejected by the operating system.

- **Defensive Error Handling:**

If file access fails due to invalid path or permissions, the system prints an error message and safely terminates execution.

Step 3: Sequential Word Processing Pipeline

- **Word Extraction Loop (`fscanf`):**

The loop `while (fscanf(fp, "%s", word) != EOF)` reads words sequentially from the file until the end-of-file condition is reached.

- **Word Count Increment:**

Each successfully detected word increments the total word counter using `word_count++`.

- **Word Length Computation:**

The function `strlen(word)` calculates the number of characters in each word.

- **Frequency Mapping:**

The corresponding index in `length_count[length]` is incremented to maintain the frequency of words based on their lengths.

Step 4: Histogram Construction Logic

- **Maximum Frequency Detection:**

The program identifies the highest word-frequency count to determine the height of the histogram.

- **Vertical Histogram Rendering:**

Nested loops print "*" symbols vertically to visually represent the frequency distribution of word lengths.

- **Axis Representation:**

A horizontal separator and corresponding word lengths are printed below the histogram for clear interpretation.

Step 5: Resource Deallocation and Process Termination

- **File Stream Closure (`fclose`):**

Calling `fclose(fp)` releases the active file resource and prevents unnecessary resource occupation.

- **Successful Program Termination:**

Executing `return 0;` signals successful completion of word analysis and histogram generation to the operating system.

6. Results:

Compiling and Profiling Environment

Validation testing was conducted in a Unix terminal environment configured with the following technical specifications:

- **Host Operating System:** Ubuntu (Latest Version) Linux Operating System.
- **Compiler Architecture:** GCC (GNU Compiler Collection) targeting standard x86_64 Linux architecture flags.
- **Compilation Command Structure:**

```

❯ Bash

gcc project1.c
./a.out words.txt

```

To thoroughly verify the implementation, the program was executed using a sample text file named `words.txt`. The terminal logs capture the exact validation sequence:

```

vinyask@vinyask-VMware-Virtual-Platform:~/Desktop/home$ gcc project1.c
vinyask@vinyask-VMware-Virtual-Platform:~/Desktop/home$ cat words.txt
Hello Guys I am working on linux with my friends
vinyask@vinyask-VMware-Virtual-Platform:~/Desktop/home$ ./a.out words.txt
Word count = 10

Word length count:
1 letter word = 1
2 letter word = 3
4 letter word = 2
5 letter word = 2
7 letter word = 2

Vertical Histogram:

  *
 * * * *
* * * * *
-----
1 2 4 5 7
vinyask@vinyask-VMware-Virtual-Platform:~/Desktop/home$

```

Diagnostic Output Inspection and Bitstream Verification

By reviewing the command sequence shown in the terminal execution profile, the behavioral properties of the system can be evaluated:

1. Compilation Step:

Executing `gcc project1.c` successfully compiles the source code into the executable binary (`a.out`) without syntax errors, confirming correct program structure and successful compilation in the Ubuntu environment.

2. Input File Verification:

Executing `cat words.txt` displays the contents of the input file:

```

HELlo Guys I am working on linux with my friends

```

This confirms that the source file contains words of varying lengths for testing the accuracy of word counting and histogram generation.

3. Execution Output Analysis:

Running `./a.out words.txt` successfully parses the input file, computes the **total word count = 10**, classifies words according to their lengths, and generates a **vertical histogram** representing the frequency distribution of word lengths.

4. Output Verification:

The generated results confirm that the program accurately processed the file contents without data omission, counted word lengths correctly, and displayed the histogram in the required

vertical format, validating the correctness of file handling, string processing, and statistical computation logic.

Boundary and Stress Case Testing

- **Empty File Analysis (0-byte Length):**

If the input file contains no textual data, the program successfully opens the file through `fopen()`, but the `while(fscanf(fp, "%s", word) != EOF)` condition fails immediately since no words are available for reading. The file stream closes safely using `fclose()`, and the program terminates without runtime faults while displaying a word count of zero.

- **Repeated Word-Length Handling:**

If multiple words of identical lengths appear consecutively in the input file, the system processes them sequentially and increments the corresponding frequency index in `length_count[]`. For example, repeated 5-letter words increase the count for `length_count[5]`, ensuring accurate histogram representation.

- **Large File Resilience:**

If a large text file is supplied, the program processes it efficiently without loading the complete content into memory. Since words are analyzed sequentially and only frequency counts are stored, memory consumption remains constant, preventing excessive resource utilization or memory overflow.

Histogram Scalability

While the current implementation generates a fixed vertical histogram using "*" characters, advanced statistical visualization tools often implement dynamic scaling techniques for improved readability when handling extremely large datasets.

In the present implementation, histogram height is determined by the maximum frequency count among word lengths. The program first identifies the highest occurrence value (`max`) and then prints rows sequentially from maximum to minimum, ensuring proportional visual representation of word-length frequency distribution.

The mathematical relationship governing histogram height is based on:

$$\text{Histogram Height} = \max(\text{Word Length Frequency})$$

Where the maximum value represents the highest occurrence count among all detected word lengths. This approach ensures accurate graphical representation while maintaining simple implementation complexity within the Linux-based C environment.

7. Conclusion:

The implementation of the **word count and histogram generation utility** satisfies all functional, structural, and algorithmic requirements specified in the project design. By using C standard file-handling and string-processing libraries instead of storing the complete file contents in memory, the utility achieves efficient execution while maintaining a constant **O(1)** auxiliary memory footprint.

The software safely handles common operational conditions such as missing files, invalid command-line arguments, and empty input files through defensive validation checks. By processing words sequentially and maintaining frequency counts using a fixed-size array, the program ensures reliable execution and accurate statistical computation without excessive memory consumption.

Terminal verification tests confirm that the utility correctly reads input files, computes total word count, classifies words according to length, and generates a vertical histogram representation accurately. The implementation demonstrates the practical application of file handling, arrays, string manipulation, and command-line interfacing concepts in a Linux-based C programming environment.

8. References:

1. The C Programming Language (1988). *The C Programming Language* (2nd ed.). Prentice Hall. (Comprehensive reference for C programming fundamentals, file handling, string processing, and command-line argument parsing.)
2. International Organization for Standardization (2011). *Information Technology – Programming Languages – C (ISO/IEC 9899:2011)*. Geneva, Switzerland: ISO. (Reference for C language standards, file input/output operations, and string manipulation behavior.)
3. Advanced Programming in the UNIX Environment (2013). *Advanced Programming in the UNIX Environment* (3rd ed.). Addison-Wesley Professional. (Reference for Linux-based file systems, command-line execution, and stream processing concepts.)
4. GNU Project. *GNU Compiler Collection (GCC) Documentation*. (Reference for compiling and executing C programs in Linux environments using GCC.)
5. IEEE Computer Society (2018). *The Open Group Base Specifications Issue 7 / IEEE Std 1003.1-2017 (POSIX.1)*. (Guidelines related to standard file handling, command-line utilities, and Linux-compatible execution behavior.)

9. Appendices:

Code:

```
#include <stdio.h>
#include <string.h>

int main(int argc, char *argv[])
{
    FILE *fp;
    char word[100];
    int word_count = 0;
    int length_count[50] = {0};
    int length, i, j;
    int max = 0;

    // Check command line argument
    if(argc != 2)
    {
        printf("Usage: ./a.out filename\n");
        return 1;
    }

    // Open file
    fp = fopen(argv[1], "r");

    // Check if file opened
    if(fp == NULL)
    {
        printf("Error opening file\n");
        return 1;
    }

    // Read words from file
    while(fscanf(fp, "%s", word) != EOF)
    {
        word_count++;

        // Find length of word
        length = strlen(word);

        // Count word length frequency
        length_count[length]++;
    }

    // Close file
    fclose(fp);

    // Print total word count
    printf("Word count = %d\n\n", word_count);

    // Print word length count
    printf("Word length count:\n");

    for(i = 1; i < 50; i++)
```

```

{
    if(length_count[i] > 0)
    {
        printf("%d letter word = %d\n", i, length_count[i]);

        // Find maximum frequency
        if(length_count[i] > max)
        {
            max = length_count[i];
        }
    }
}

// Print vertical histogram
printf("\nVertical Histogram:\n\n");

for(i = max; i >= 1; i--)
{
    for(j = 1; j < 50; j++)
    {
        if(length_count[j] >= i)
            printf(" * ");
        else if(length_count[j] > 0)
            printf(" ");
    }

    printf("\n");
}

// Print x-axis
for(i = 1; i < 50; i++)
{
    if(length_count[i] > 0)
        printf("---");
}

printf("\n");

// Print word lengths
for(i = 1; Word Count and Vertical Histogram Generation Using File Handling in C
on Linux i < 50; i++)
{
    if(length_count[i] > 0)
        printf("%2d ", i);
}

printf("\n");

return 0;

```

